

Neovim Command & Shortcut Reference

Built for your setup: lazy.nvim + blink.cmp + LSP (mason) + Treesitter + Telescope + Oil + Gitsigns + Neogit + Diffview + Lazygit + Conform + nvim-lint + which-key + mini.nvim

This is organized plugin-by-plugin, from core Neovim motions up to your git and LSP workflow. Each row is tagged with a level: **Core** (must-know, use constantly), **Easy** (simple, low-risk, quick to learn), **Medium** (useful but needs a bit of muscle memory), **Hard** (powerful but easy to misuse or forget - worth deliberately practicing).

Important note on leader-key mappings: Lines marked **[confirmed]** in the notes are taken directly from your lazy.nvim profiling output (lazygit.nvim <leader>gg, diffview.nvim <leader>gh/gH/gc/gd, neogit <leader>gs, telescope <leader>ff/fq/fb/fh, conform <leader>cf). Everything else is the plugin's **default** mapping - your actual config may remap these. Run `:Telescope keymaps` or press <leader> and wait (which-key will pop up the real list) to check your exact bindings.

Contents

1. Core Neovim - modes, motions, editing
2. Search, replace & multi-file (core)
3. Buffers, windows, tabs (core)
4. Marks, registers & macros
5. Oil.nvim - file explorer
6. Telescope.nvim - fuzzy finder [confirmed keymaps]
7. LSP (nvim-lspconfig + mason + mason-lspconfig)
8. blink.cmp - completion
9. nvim-treesitter
10. conform.nvim - formatting [confirmed keymaps]
11. nvim-lint - linting
12. gitsigns.nvim - inline git
13. lazygit.nvim [confirmed keymap]
14. neogit [confirmed keymap]
15. diffview.nvim [confirmed keymaps]
16. mini.nvim - common modules
17. which-key.nvim
18. lazy.nvim - plugin manager
19. Efficient workflows - combining it all

1. Core Neovim — Modes & Motions

The absolute foundation. If any of these feel shaky, drill them before worrying about plugins.

Key / Command	What it does	Level
i / a	Insert before / after cursor	Core
I / A	Insert at start / end of line	Core
o / O	New line below / above, enter insert	Core
Esc / Ctrl-[	Back to Normal mode	Core
v / V / Ctrl-v	Visual / Visual Line / Visual Block	Core
h j k l	Left, down, up, right	Core
w / b / e	Next word / back word / end of word	Core
W / B / E	Same, but WORD (whitespace-delimited)	Easy
0 / ^ / \$	Start of line / first non-blank / end of line	Core
gg / G	Top of file / bottom of file	Core
{n}G or :n	Go to line n	Easy
%	Jump to matching bracket/paren	Easy
f{c} / F{c}	Jump to next / previous char c on line	Medium
t{c} / T{c}	Jump till before / after char c	Medium
; / ,	Repeat last f/t forward / backward	Medium
Ctrl-d / Ctrl-u	Half page down / up	Core
Ctrl-f / Ctrl-b	Full page down / up	Easy
zz / zt / zb	Center / top / bottom cursor line on screen	Medium
H / M / L	Top / middle / bottom of visible screen	Medium

Editing & text objects

Key / Command	What it does	Level
x / X	Delete char under / before cursor	Core
dd / yy	Delete (cut) line / yank (copy) line	Core
p / P	Paste after / before cursor	Core
u / Ctrl-r	Undo / redo	Core
r{c}	Replace single char with c	Easy
cc / C	Change whole line / to end of line	Easy
cw / ciw	Change to end of word / change inner word	Medium
dw / diw	Delete to end of word / delete inner word	Medium
di" / da"	Delete inside / around quotes (also ([{ etc.)	Medium
ci(/ ca(	Change inside / around parens (also { [< etc.)	Medium
>> / <<	Indent / unindent line	Easy
= / gg=G	Auto-indent selection / whole file	Medium
. (dot)	Repeat last change — huge productivity boost	Hard
J	Join current line with next	Easy
~	Toggle case of char under cursor	Easy
guu / gUU	Lowercase / uppercase current line	Medium
Ctrl-a / Ctrl-x	Increment / decrement number under cursor	Medium

Key / Command	What it does	Level
v_p	In visual mode: paste over selection without losing yank register	Hard

2. Search, Replace & Multi-file (core)

Key / Command	What it does	Level
<code>/pattern Enter</code>	Search forward	Core
<code>?pattern Enter</code>	Search backward	Easy
<code>n / N</code>	Repeat search same / opposite direction	Core
<code>* / #</code>	Search word under cursor forward / backward	Medium
<code>:%s/old/new/g</code>	Replace all occurrences in file	Core
<code>:%s/old/new/gc</code>	Replace all, with confirmation per match	Medium
<code>:s/old/new/g</code>	Replace all occurrences on current line only	Easy
<code>:'<,'>s/old/new/g</code>	Replace within visual selection	Medium
<code>:vimgrep /pat/ **/*.c</code>	Search pattern across files into quickfix list	Hard
<code>:copen / :cclose</code>	Open / close quickfix list	Medium
<code>:cnext / :cprev</code>	Jump to next / previous quickfix entry	Medium

3. Buffers, Windows & Tabs (core)

Key / Command	What it does	Level
<code>:e file</code>	Open (edit) a file	Core
<code>:w / :w file</code>	Save / save as	Core
<code>:q / :q!</code>	Quit / quit without saving	Core
<code>:wq / :x</code>	Save and quit	Core
<code>:bn / :bp</code>	Next / previous buffer	Easy
<code>:ls or :buffers</code>	List open buffers	Easy
<code>:b {N or name}</code>	Jump to buffer by number/name	Easy
<code>:bd</code>	Delete (close) current buffer	Easy
<code>Ctrl-w s / Ctrl-w v</code>	Split window horizontally / vertically	Core
<code>Ctrl-w h/j/k/l</code>	Move focus between splits	Core
<code>Ctrl-w q</code>	Close current split	Easy
<code>Ctrl-w =</code>	Equalize split sizes	Easy
<code>Ctrl-w > / <</code>	Resize split wider / narrower	Medium
<code>:tabnew / gt / gT</code>	New tab / next tab / previous tab	Medium

4. Marks, Registers & Macros

Key / Command	What it does	Level
<code>m{a}</code>	Set mark a at cursor position	Medium
<code>'a</code> or <code>`a</code>	Jump to mark a (line / exact position)	Medium
<code>``</code>	Jump back to position before last jump	Medium
<code>"a y y</code> then <code>"a p</code>	Yank line into register a, paste from register a	Hard
<code>:reg</code>	Show contents of all registers	Medium
<code>qa ... q</code>	Record macro into register a, stop recording	Hard
<code>@a</code>	Replay macro a once	Hard
<code>@@</code>	Replay last-used macro again	Hard
<code>5@a</code>	Replay macro a five times	Hard
<code>:g/pattern/normal @a</code>	Run macro a on every line matching pattern	Hard

5. Oil.nvim — File Explorer

Oil edits your filesystem like a normal buffer. Defaults shown below - your config may have remapped the launch key (commonly `<leader>e` or `-`).

Key / Command	What it does	Level
<code>-</code> (in a file buffer)	Open Oil in the parent directory (default map)	Core
<code><CR></code>	Open file/dir under cursor	Core
<code>Ctrl-s</code> / <code>Ctrl-v</code>	Open in horizontal / vertical split	Medium
<code>Ctrl-t</code>	Open in new tab	Medium
<code>g.</code>	Toggle showing hidden files	Easy
<code>g?</code>	Show Oil keymap help	Easy
<code>gs</code>	Change sort order	Medium
<code>-</code> (inside Oil)	Go up one directory	Easy
<code>_</code>	Open Neovim's cwd in Oil	Medium
<code>:w</code> (inside Oil)	Apply changes: create/rename/delete files by editing the buffer as text	Hard

6. Telescope.nvim — Fuzzy Finder

The four core mappings below are read straight from your lazy.nvim keymap triggers (<leader>ff / fg / fb / fh). [confirmed]

Key / Command	What it does	Level
<leader>ff	Find files in cwd [confirmed]	Core
<leader>fg	Live grep (search text across project) [confirmed]	Core
<leader>fb	List open buffers [confirmed]	Core
<leader>fh	Search help tags [confirmed]	Easy

Inside any Telescope picker window:

Key / Command	What it does	Level
Ctrl-n / Ctrl-p (or j/k)	Move down / up the result list	Core
Enter	Open selection in current window	Core
Ctrl-x / Ctrl-v / Ctrl-t	Open selection in split / vsplit / tab	Medium
Tab / Shift-Tab	Toggle multi-select on entry / move up selecting	Medium
Ctrl-q	Send all results to quickfix list	Hard
Esc (in insert) / q (normal)	Close picker	Easy
Ctrl-u / Ctrl-d	Scroll preview pane up / down	Medium
:Telescope keymaps	Search your ACTUAL current keymaps — use this to verify anything in this PDF	Hard
:Telescope live_grep	Same as <leader>fg, callable directly	Easy
:Telescope oldfiles	Recently opened files (not bound by default here)	Medium

7. LSP — nvim-lspconfig + mason.nvim + mason-lspconfig

Your config uses the modern `vim.lsp.enable()` API. These are the standard buffer-local LSP mappings most configs set up on `LspAttach`; check your config for exact binds.

Key / Command	What it does	Level
K	Hover docs for symbol under cursor	Core
gd	Go to definition	Core
gD	Go to declaration	Medium
gr	Go to references (list all usages)	Core
gI	Go to implementation	Medium
<leader>rn	Rename symbol project-wide	Core
<leader>ca	Code actions (quick fixes, refactors)	Core
[d /]d	Jump to previous / next diagnostic	Core
<leader>e or vim.diagnostic.open_float()	Show diagnostic message in a float	Medium
Ctrl-k (insert mode)	Signature help while typing arguments	Medium
:LspInfo	Show attached LSP clients for this buffer	Medium
:Mason	Open Mason UI to browse/install LSPs, linters, formatters	Core
:MasonInstall {tool}	Install a specific tool (e.g. clangd, pyright)	Easy
:MasonUpdate	Update all installed Mason packages	Easy
:checkhealth vim.lsp	Diagnose LSP setup problems	Hard

8. blink.cmp — Completion

Key / Command	What it does	Level
Start typing	Completion menu appears automatically	Core
Ctrl-Space	Manually trigger / open completion menu	Core
Tab / Ctrl-n	Select next completion item	Core
Shift-Tab / Ctrl-p	Select previous completion item	Core
Enter or Ctrl-y	Accept selected completion	Core
Ctrl-e	Close/cancel completion menu	Easy
Tab (inside snippet)	Jump to next snippet placeholder (friendly-snippets)	Medium
Shift-Tab (inside snippet)	Jump to previous snippet placeholder	Medium
Ctrl-u / Ctrl-d	Scroll documentation preview of item	Medium

9. nvim-treesitter

Powers syntax highlighting, indentation, and structural (incremental) selection.

Key / Command	What it does	Level
<code>:TSUpdate</code>	Update all installed parsers	Easy
<code>:TSInstall {lang}</code>	Install a parser for a language	Easy
<code>:TSInstallInfo</code>	List installed/available parsers	Medium
<code>Ctrl-space</code> (in normal mode, if mapped)	Start incremental selection at cursor node	Hard
<code>Ctrl-space</code> (repeated)	Expand selection to next larger syntax node	Hard
<code>Backspace</code> (while selecting)	Shrink selection to previous node	Hard
<code>via text objects: daf / dif</code>	Delete around/inside function (if textobjects module enabled)	Hard
<code>:checkhealth nvim-treesitter</code>	Diagnose parser/highlighting issues	Medium

10. conform.nvim — Formatting

Your config loads conform on BufWritePre (auto-format on save) and binds <leader>cf to format-on-demand. [confirmed]

Key / Command	What it does	Level
<code><leader>cf</code>	Format current buffer on demand [confirmed]	Core
<code>:w</code>	Triggers auto-format via BufWritePre before saving [confirmed]	Core
<code>:ConformInfo</code>	Show which formatters are attached to this buffer [confirmed]	Medium
<code>:lua require('conform').format()</code>	Format manually from the command line / a keymap you write	Medium

11. nvim-lint

Loads on BufReadPost in your config — runs linters in the background and reports via the diagnostics API.

Key / Command	What it does	Level
(automatic)	Runs configured linters after reading a buffer and on save	Core
[d /]d	Jump between lint diagnostics (same as LSP diagnostics)	Core
vim.diagnostic.open_float()	View the lint message under cursor	Medium
:lua require('lint').tryLint()	Manually re-run linters on current buffer	Medium
:checkhealth	Check nvim-lint / linter executables are found on PATH	Medium

12. gitsigns.nvim — Inline Git

Loads on BufReadPre. Shows added/changed/removed lines in the sign column. Mappings below are the plugin's documented defaults — confirm your leader prefix.

Key / Command	What it does	Level
]c / [c	Jump to next / previous git hunk	Core
<leader>hs	Stage hunk under cursor	Core
<leader>hr	Reset (undo) hunk under cursor	Medium
<leader>hS	Stage entire buffer	Medium
<leader>hu	Undo last stage hunk	Medium
<leader>hp	Preview hunk diff in a float	Easy
<leader>hb	Blame current line (float)	Medium
<leader>tb	Toggle inline blame virtual text	Medium
<leader>hd	Diff current buffer against index	Medium
ih (text object)	Select current hunk, e.g. dih to delete it	Hard

13. lazygit.nvim

<leader>gg opens the full lazygit TUI inside a floating Neovim window. [confirmed]

Key / Command	What it does	Level
<leader>gg	Open LazyGit floating window [confirmed]	Core
q	Close LazyGit and return to Neovim	Easy

Inside LazyGit itself (standard lazygit keys, not Neovim):

Key / Command	What it does	Level
Tab / Shift-Tab	Move between panels (status, files, branches, commits, stash)	Medium
Space	Stage / unstage file or hunk under cursor	Core
c	Commit staged changes (opens message prompt)	Core
P	Push	Core
p	Pull	Core
b	Branches panel	Medium
d (in file view)	Discard changes to file	Hard
Enter (on file)	View diff / stage hunks individually	Medium
x	Open command/help menu	Easy

14. neogit

`<leader>gs` opens Neogit's status buffer (Magit-style git porcelain, native Neovim buffer). [confirmed]

Key / Command	What it does	Level
<code><leader>gs</code>	Open Neogit status buffer [confirmed]	Core
<code>:Neogit</code>	Same, callable directly [confirmed]	Easy

Inside the Neogit status buffer (defaults):

Key / Command	What it does	Level
Tab	Expand/collapse section (diff hunks, files)	Easy
s	Stage file/hunk under cursor	Core
u	Unstage file/hunk under cursor	Core
c	Open commit menu (cc = commit)	Core
p	Open pull menu	Medium
P	Open push menu	Medium
b	Open branch menu (checkout/create/delete)	Medium
l	Open log menu	Medium
x	Discard changes under cursor	Hard
q	Close Neogit buffer	Easy
?	Show all available commands in current context	Easy

15. diffview.nvim

Your keymaps <leader>gh / gH / gc / gd are wired to diffview's commands. [confirmed]

Key / Command	What it does	Level
<leader>gh	File history for current file (:DiffviewFileHistory %) [confirmed]	Core
<leader>gH	Full repo history / branch history [confirmed]	Medium
<leader>gc	Likely: close diffview or toggle file panel [confirmed - verify exact action]	Medium
<leader>gd	Open diff view (:DiffviewOpen) [confirmed]	Core
:DiffviewOpen	Open diff of working tree vs HEAD	Core
:DiffviewOpen main...HEAD	Diff current branch against main	Medium
:DiffviewClose	Close diffview	Easy
:DiffviewFileHistory	Browse commit history for repo or a path	Medium
:DiffviewToggleFiles	Toggle the file panel sidebar	Easy

Inside a diffview file panel:

Key / Command	What it does	Level
j / k	Move between files in the panel	Easy
Enter	Open diff for file under cursor	Easy
Tab / Shift-Tab	Next / previous file diff	Medium
co / ct / cb	Choose OURS / THEIRS / BASE version during a merge conflict	Hard
g?	Show help for current view	Easy

16. mini.nvim

mini.nvim ships dozens of independent modules (pairs, surround, statusline, icons, ai, comment, etc). Your profile only shows 'mini.nvim' loading as a group, so check your config for exactly which modules you enabled. Below are the most commonly-used ones if you have them on.

Key / Command	What it does	Level
<code>sa{motion}{char}</code>	mini.surround: Add surround, e.g. <code>saiw"</code> surrounds word with quotes	Hard
<code>sd{char}</code>	mini.surround: Delete surround, e.g. <code>sd"</code> removes quotes	Medium
<code>sr{old}{new}</code>	mini.surround: Replace one surround with another	Hard
<code>gcc</code>	mini.comment: Toggle comment on current line	Core
<code>gc (visual)</code>	mini.comment: Toggle comment on selection	Core
<code>(/)</code>	mini.ai / built-in: Move by sentence (if mini.ai extends text objects)	Medium
<code>(auto)</code>	mini.pairs: Auto-close brackets/quotes as you type	Easy
<code>(auto)</code>	mini.statusline: Shows mode, git branch, diagnostics, filetype in statusline	Easy
<code>(auto)</code>	mini.icons: Provides filetype icons used by Oil, Telescope, etc.	Easy
<code>:lua =require('mini.deps').config</code>	Check which mini modules are actually configured	Hard

17. which-key.nvim

Key / Command	What it does	Level
<code><leader> (then wait)</code>	Pops up a menu of every mapping starting with leader — your live cheat sheet	Core
<code>g (then wait)</code>	Shows all g-prefixed mappings (gd, gc, gg, etc.)	Easy
<code>z (then wait)</code>	Shows all z-prefixed mappings (zz, zt, folds, spelling)	Medium
<code>:WhichKey</code>	Open the which-key popup manually for the current mode	Easy
<code>:WhichKey <leader>g</code>	Show only mappings under a specific prefix	Medium

18. lazy.nvim — Plugin Manager

Key / Command	What it does	Level
<code>:Lazy</code>	Open the plugin manager UI (what generated your pasted output)	Core
<code>(inside :Lazy) i</code>	Install missing plugins	Easy
<code>(inside :Lazy) u</code>	Update all plugins to latest	Easy
<code>(inside :Lazy) x</code>	Clean: remove plugins no longer in your config	Medium
<code>(inside :Lazy) s</code>	Sync (install+clean+update in one go)	Medium
<code>(inside :Lazy) p</code>	Show profiling info (load times per plugin) — this is what you pasted	Medium
<code>(inside :Lazy) l</code>	Show changelog for a selected plugin	Easy
<code>:Lazy log</code>	Git log of recent plugin updates	Medium
<code>:Lazy restore</code>	Restore plugin versions to lockfile state	Hard

19. Efficient Workflows — Putting It Together

Editing a file end-to-end:

<leader>ff to find it → edit with normal-mode motions + dot-repeat → gd/gr to jump around via LSP → <leader>ca to fix a diagnostic → :w triggers conform auto-format → <leader>hs to stage the hunk in gitsigns.

Committing:

Fast/simple: <leader>gg for LazyGit's TUI, stage with Space, c to commit, P to push.

Native buffer style: <leader>gs for Neogit status, s to stage, c to open commit menu.

Reviewing changes before committing:

<leader>gd (DiffviewOpen) to see a full working-tree diff, or <leader>gh for file history on the current file.

Exploring a new/unfamiliar project:

<leader>ff / <leader>fg to search files and text → <leader>fb to jump between open buffers → use - to drop into Oil when you need to look at directory structure or bulk-rename files.

Learning what you don't remember:

Press <leader> and just wait — which-key will show you every real mapping active right now, which is more reliable than any static PDF (including this one) since it reflects your actual live config.

Suggested drilling order if you want to get faster deliberately:

1) Core motions + dot-repeat until they're automatic → 2) Telescope find/grep until you stop reaching for a file tree → 3) LSP gd/gr/<leader>ca until you stop alt-tabbing to docs → 4) One git workflow (pick LazyGit OR Neogit, not both) until staging/committing is muscle memory → 5) Macros (qa...q, @a) once the above feel boring - that's when repetitive edits start showing up often enough to be worth it.

Generated for your current lazy.nvim setup based on the plugin profiling output you shared. Verify leader-prefixed mappings that aren't marked [confirmed] against your own config or live which-key popups, since defaults get remapped often.